

BlueFeather 所見 — 基本設計（R1）

合格ライン 80.0点 / status: ok

いま57点。合格ラインの80点まで、あと一步というところです。

サンプルさん、ここまでの量を、画面、状態遷移、機能、テストケース、デシジョンテーブルに分けて整理したのは立派です。特に、入力範囲の境界や無効遷移まで見に行こうとしているところに、仕様をきちんとテスト条件へ落とそうとする真面目さが出ています。

いま57点。合格ラインの80点まで、あと一步というところです。

- ・成果物の骨格はできています。次は、仕様に書かれている制約が『画面で入力できない』ものなのか『処理時にエラーになる』ものなのかを分けると、テスト設計の精度が上がります。
- ・共通機能は一箇所に整理されていますが、テスト条件としては横断的に効いてきます。エラー通知、通信エラー、ログアウト、戻る、入力中表示を観点表にして、各画面へ展開するとよいです。
- ・テストケースの期待値は、遷移や表示有無だけでなく、計算結果や保持データの中身まで具体化すると、実行者による判断ぶれを減らせます。

まずは『入力制御と処理エラーを分ける』『共通機能と通信エラーを横断観点にする』『期待値を具体的な数値まで書く』の三つから、いってみませんか。

技法レコメンド

対象	おすすめ技法	理由
人数、金額、支払割合、備考文字数、開催日、開催時刻などの入力範囲	同値分割・境界値分析	有効範囲と無効範囲が仕様で明確に示されていますので、代表値だけでなく境界の直前・境界値・境界の直後を整理すると、今あるケースをより説明しやすくなります。
ログイン条件、登録可否、ジャスPayボタン表示有無などの条件分岐	デシジョンテーブル	複数条件の組み合わせで結果が変わるところは、今の方向性どおりデシジョンテーブルが合っています。次は『入力制御で発生しない組み合わせ』を除外条件として明記すると、表が引き締まります。
ログイン画面から各画面への遷移、戻る、ログアウト、再起動後の初期画面	状態遷移テスト	画面遷移が品質に直結するアプリですので、状態遷移テストは有効です。有効遷移と無効遷移を分け、さらにログアウトやアプリ再起動を状態として扱うと、共通機能まで自然に拾えます。
通信エラー、サーバ異常、QRコード生成失敗、履歴取得失敗	エラー推測・フォールトインJECTION	外部サービスやサーバ通信は、正常系だけでは利用時の困りごとを拾いきれません。通信不可やタイムアウトを意図的に起こす観点を置くと、エラー通知と処理中止の仕様を確認しやすくなります。
割り勘計算式、百円単位切り上げ、お釣り算出	パラメータ化テスト・期待値テーブル	計算機能は『表示されること』だけでなく、具体的な数値が合っていることが大切です。入力値と期待される自分側金額、相手側金額、お釣りを表にしておくと、レビューもしやすくなります。

ループリック項目別

項目	score/max	根拠
----	-----------	----

technique_selection	3/4	状態遷移表、無効遷移を含めた遷移確認、デシジョンテーブル、境界値を意識した入力値選定が入っていて、基本設計としての技法選択はかなり筋が通っています。割り勘計算、ログイン、登録画面のように条件分岐や入力範囲がある機能に対して、技法を使い分けようとしている姿勢が見えます。
condition_derivation	2/4	ログイン、割り勘計算、登録、履歴表示については、主要な正常系と境界付近の条件がかなり拾えています。特に人数、金額、日付、備考文字数の境界を見に行っている点は、仕様を読んで条件に落とそうとした跡がはっきりあります。一方で、共通機能や外部通信まわり、表示制御条件の導出がまだ薄く、重要な故障モードが残りそうです。
viewpoint_mapping	2/4	テストスイートに画面、機能、観点の列があり、どの画面のどの機能を見ているかは追える形になっています。デシジョンテーブルや状態遷移表も別シートに分かれていて、成果物として読み手に見せようとする構造があります。次は、要求・仕様・技法・テスト条件・テストケースのつながりをもう一段明示すると、設計として説明しやすくなります。
design_consistency	2/4	画面一覧、状態遷移、機能一覧、テストスイート、デシジョンテーブルを同じ成果物の中に揃えており、全体を一つの設計としてまとめようとしている点は良いです。私なんてこの歳ですから、こういう大きな表を作ると表記ゆれや前提ずれをよく起こしてきました。今回も、成果物間のつながりを少し整えるだけで、かなり読みやすくなりそうです。

QA4AI（レビューの点検）

ルール遵守: 違反なし

根拠の妥当性

指摘	根拠	理由
成果物の骨格はできています。次は、仕様に書かれている制約が『画面で入力できない』ものなのか『処理時にエラーになる』ものなのかを分けると、テスト設計の精度が上がります。	あり	成果物には概要、画面一覧、状態遷移一覧、機能一覧、テストスイート一覧、デシジョンテーブル一覧、一般用語があり、テスト設計成果物としての構成は存在する。また機能一覧では、ユーザID・パスワードはテキストフィールド設定により文字種・文字数が制限される、人数・金額は数字のみ入力可能など『画面で入力できない』制約が書かれている一方、範囲外の人数・金額では『入力された数字が不正です』を表示するなど『処理時にエラーになる』制約も書かれている。テストケースではTC-006、TC-007、TC-010、TC-011などで、本来入力制限される可能性のある値を入力して表示されることを期待しているため、両者の区別を明確化すべきという指摘には根拠がある。
共通機能は一箇所に整理されていますが、テスト条件としては横断的に効いてきます。エラー通知、通信エラー、ログアウト、戻る、入力中表示を観点表にして、各画面へ展開するとよいです。	あり	機能一覧の『共通機能』に、エラー通知、戻るボタン、入力中の表示、ログアウト処理、通信エラー処理がまとめて記載されている。これらは複数画面・複数機能に関係する内容であり、テストスイートでは戻るボタンや一部エラー通知は扱われているが、通信エラー処理、ログアウト処理、入力中表示などが各画面へ網羅的に展開されているとは読み取れないため、横断観点として整理するという指摘には根拠がある。

テストケースの期待値は、遷移や表示有無だけでなく、計算結果や保持データの中身まで具体化すると、実行者による判断ぶれを減らせます。	あり	テストスイートには、TC-019、TC-020、TC-023、TC-024、TC-027、TC-028、TC-031、TC-032などで期待値が『計算結果が表示されること』にとどまり、具体的な計算結果の数値が書かれていないケースがある。またTC-042、TC-044、TC-060では『遷移前の状態が保持されていること』とあるが、どの入力値・計算結果・選択状態を確認するかが具体化されていない。一方でTC-043やTC-057のように具体的な計算結果を書いているケースもあるため、全体として具体化の余地があるという指摘には根拠がある。
デシジョンテーブル、状態遷移表、テストケースの対応関係をIDでつなぐと、なぜそのケースが必要なのかを読み手が追いやすくなります。	あり	成果物には状態遷移一覧の遷移テストケース番号、テストスイート一覧のTC番号、デシジョンテーブル一覧の行番号がそれぞれ存在する。しかし、例えばテストケースTC-019がデシジョンテーブルのどの行に対応するか、TC-035などが状態遷移表のどの遷移に対応するかを示す対応IDや参照列は記載されていない。そのため、対応関係をIDでつなぐと追跡しやすくなるという指摘には根拠がある。

LLM-as-judge（各0～4）

具体性	実行可能性	ペルソナ遵守	点数非漏洩
4	4	4	4

総評:

承認と具体的な改善提案のバランスがよく、読み手が次に何を直せばよいか分かりやすい所見です。